

## Renforcement Java – TD

---

### Exercice 1

1. Soit l'expression : `2 >= 3 || 12%2 == 0`
  - (a) C'est une faute de syntaxe.
  - (b) Son résultat est de type int.
  - (c) Son résultat est de type boolean.
  - (d) Son résultat est 6.
  - (e) Son résultat est vrai.
  - (f) Son résultat est faux.
2. Soit l'expression : `0 < 2 < 5`
  - (a) C'est une faute de syntaxe.
  - (b) Son résultat est de type int.
  - (c) Son résultat est de type boolean.
  - (d) Son résultat est 6.
  - (e) Son résultat est vrai.
  - (f) Son résultat est faux.
3. Soit l'expression : `2/5`
  - (a) C'est une faute de syntaxe.
  - (b) Son résultat est de type int.
  - (c) Son résultat est de type double.
  - (d) Son résultat est 0.4.
  - (e) Son résultat est 0.
  - (f) Cela provoque une erreur pendant l'exécution.
4. Soit l'expression : `(double) (2/5)`
  - (a) C'est une faute de syntaxe.
  - (b) Son résultat est de type int.
  - (c) Son résultat est de type double.
  - (d) Son résultat est 0.4.

- (e) Son résultat est 0.0.
- (f) Cela provoque une erreur pendant l'exécution.

5. Soit l'expression : `((double) 2)/5`

- (a) C'est une faute de syntaxe.
- (b) Son résultat est de type `int`.
- (c) Son résultat est de type `double`.
- (d) Son résultat est 0.4.
- (e) Son résultat est 0.0.
- (f) Cela provoque une erreur pendant l'exécution.

6.

- (a) Une donnée de type `int` peut être négative.
- (b) Une donnée de type `int` peut valoir 1000000000 (un milliard).
- (c) Une donnée de type `int` peut valoir 100000000000 (cent milliards).
- (d) Une donnée de type `long` peut valoir 10000000000 (un milliard).
- (e) Une donnée de type `long` peut valoir 1000000000000 (cent milliards).
- (f) Une donnée de type `int` peut valoir 0.5 (un demi).
- (g) Une donnée de type `long` peut valoir 0.5 (un demi).

7.

- (a) Une donnée de type `double` peut être négative.
- (b) Une donnée de type `double` peut valoir 1000000000 (un milliard).
- (c) Une donnée de type `double` peut valoir 100000000000 (cent milliards).
- (d) Une donnée de type `double` peut valoir 0.5 (un demi).
- (e) Une donnée de type `double` peut être nulle.

8. Considérons le type `Couleur` et les fonctions `estChaude` et `estFroide` ainsi définies :

```
enum Couleur {rouge, orange, jaune, vert, bleu, violet};

static boolean estChaude(Couleur c){
    return c == Couleur.rouge || c == Couleur.orange || c == Couleur.jaune;
}

static boolean estFroide(Couleur c){
    return !estChaude(c);
}
```

- (a) L'expression `estChaude(Couleur vert)` est une erreur de syntaxe.
- (b) L'expression `estChaude(Couleur.vert)` vaut `true` (vrai).

- (c) L'expression `estChaude(Couleur.vert)` vaut `false` (faux).
- (d) L'expression `estChaude(Couleur.vert)` vaut `0` (entier nul).
- (e) L'expression `estFroide(Couleur.vert)` vaut `false` (faux).
- (f) L'expression `estFroide(Couleur.vert)` vaut `true` (vrai).
- (g) L'expression `estChaude(Couleur.vert)` est une erreur de syntaxe.
- (h) L'expression `estChaude(Couleur.rouge)` est une erreur de syntaxe.
- (i) L'expression `estChaude(Couleur.indigo)` est une erreur de syntaxe.
- (j) L'expression `estChaude(Couleur.rouge)` vaut `true` (vrai).

## Exercice 2

On considère le programme suivant :

```
class TestLectures{
    public static void main(String[] arg){
        int x = 12;
        String s = "coucou";
        x = Lecture.unEntier();
        s = Lecture.chaine();
        System.out.print("x = "+x);
        System.out.println(" s = "+s);
    }
}
```

Au moment de l'exécution, l'utilisateur entre au clavier : 56 toto

1. Il ne se passe rien et le programme termine son exécution.
2. Il ne se passe rien et le programme ne se termine pas.
3. Il y a une erreur pendant l'exécution.
4. Le programme affiche : x = 56 s = toto
5. Le programme affiche : x = 12 s = coucou  
Au moment de l'exécution, l'utilisateur entre au clavier : tutu toto
6. Il ne se passe rien et le programme termine son exécution.
7. Il ne se passe rien et le programme ne se termine pas.
8. Il y a une erreur pendant l'exécution.
9. Le programme affiche : x = tutu s = toto
10. Le programme affiche : x = 12 s = coucou  
Au moment de l'exécution, l'utilisateur entre au clavier : 56 95
11. Il ne se passe rien et le programme termine son exécution.
12. Il ne se passe rien et le programme ne se termine pas.
13. Il y a une erreur pendant l'exécution.
14. Le programme affiche : x = 56 s = 95
15. Le programme affiche : x = 12 s = coucou  
Au moment de l'exécution, l'utilisateur n'entre rien au clavier.
16. Il ne se passe rien et le programme termine son exécution.
17. Il ne se passe rien et le programme ne se termine pas.
18. Il y a une erreur pendant l'exécution.
19. Le programme affiche : x = 56 s = 95
20. Le programme affiche : x = s =

## Exercice 3

On considère le programme suivant :

```
class TestFonctions{
    static int x;
    static int plusUn(int k){
        // ???
        return k + 1;
    }
    static void affichePlusUn(int x){
        // ???
        System.out.print(x + 1);
    }
    static void ajouteUnAX(){
        // ???
        x = x + 1;
    }
    public static void main(String[] arg){
        x = 12;
        System.out.print("x = " + x);
        System.out.print(" 6+1 = ");
        affichePlusUn(6);
        System.out.print(" x = " + x);
        ajouteUnAX();
        System.out.print(" x = " + x);
        System.out.println(" 8+1 = " + plusUn(8));
    }
}
```

1. Commentaire de spécification de plusUn : // résultat : k+1
2. Commentaire de spécification de plusUn : // effet : ajoute 1 à k
3. Commentaire de spécification de plusUn : // effet : affiche k+1
4. Commentaire de spécification de plusUn : // effet : ajoute 1 à x
5. Commentaire de spécification de affichePlusUn : // résultat : x+1
6. Commentaire de spécification de affichePlusUn : // effet : ajoute 1 à k
7. Commentaire de spécification de affichePlusUn : // effet : affiche x+1
8. Commentaire de spécification de affichePlusUn : // effet : ajoute 1 à x
9. Commentaire de spécification de ajouteUnAX : // résultat : x+1
10. Commentaire de spécification de ajouteUnAX : // effet : ajoute 1 à x
11. Commentaire de spécification de ajouteUnAX : // effet : affiche x+1
12. La première instruction du main, `x = 12;`, affecte la variable globale `x`.
13. La première instruction du main, `x = 12;`, donne la valeur 12 au paramètre `x` de la fonction `affichePlusUn`.

14. L'instruction du main `affichePlusUn(6);` affiche 7.
15. L'instruction du main `affichePlusUn(6);` donne la valeur 7 à la variable globale `x`.
16. Le programme affiche `x = 12` `6+1 = 7` `x = 7` `x = 8` `8+1 = 9`.
17. Le programme affiche `x = 12` `6+1 = 7` `x = 12` `x = 13` `8+1 = 9`.
18. Le programme affiche `x = 12` `6+1 = 7` `x = 13` `x = 14` `8+1 = 9`.

## Exercice 4

### Question 4.1

A propos du choix des identificateurs, on peut affirmer :

1. Il faut utiliser les noms les plus courts possibles car cela réduit la taille des programmes et les rend plus lisibles.
2. Dans la norme usuelle, les noms de classe commencent par une majuscule et les noms des données et des fonctions commencent par une minuscule.
3. Il faut utiliser les noms les plus signifiants possibles, même si cela conduit à des noms longs, car cela remplace avantageusement de nombreux commentaires et rend les programmes plus lisibles.
4. On a le droit d'utiliser `villeDeMilleHabitants` comme identificateur.
5. On a le droit d'utiliser `ville De Mille Habitants` comme identificateur.
6. On a le droit d'utiliser `villeDe1000habitants` comme identificateur.
7. On a le droit d'utiliser `1000habitants` comme identificateur.
8. On a le droit d'utiliser `classe` comme identificateur.
9. On a le droit d'utiliser `class` comme identificateur.

### Question 4.2

On considère le programme suivant :

```
class QCMLocalGlobal{
    static int x;
    static int plusK(int x, int k){
        int resul = x + k;
        return resul;
    }
    static void ajouteK(int k){
        int resul = x + k;
        x = resul;
    }
    public static void main(String[] arg){
        x = 56;
        int k = 12;
        int troisPlusK = plusK(3, k);
        System.out.print("x = " + x + " troisPlusK = " + troisPlusK);
        ajouteK(10);
        System.out.println(" x = " + x + " troisPlusK = " + troisPlusK);
    }
}
```

1. Dans le main, l'instruction `x = 56;` donne la valeur 56 au paramètre x de la fonction `plusK`.

2. Dans le main, l'instruction `x = 56;` donne la valeur 56 à la variable globale `x`.
3. Dans l'instruction `int resul = x+k;` de la procédure `ajouteK`, `x` désigne la variable locale `x` du main.
4. Dans l'instruction `int resul = x+k;` de la procédure `ajouteK`, `x` désigne la variable globale `x`.
5. L'identificateur `resul` désigne la même variable dans la fonction `plusK` et dans la procédure `ajouteK`.
6. Ce programme affiche : `x = 56 troisPlusK = 15 x = 66 troisPlusK = 15`
7. Ce programme affiche : `x = 56 troisPlusK = 68 x = 13 troisPlusK = 68`

### Question 4.3

1. Une application est généralement constituée de plusieurs classes.
2. Une application doit être constituée d'une seule classe.
3. Il est bon de regrouper dans une même classe les choses concernant un même thème.
4. La classe `Math` de la bibliothèque standard regroupe les fonctions et les constantes concernant les opérations numériques ("mathématiques").
5. Il est impossible d'utiliser une fonction d'une classe `A` depuis une autre classe `B`.
6. Pour utiliser une chose statique appelée `f` définie dans une classe `A` depuis une autre classe `B`, il faut la désigner par `A(f)`.
7. Pour utiliser une chose statique appelée `f` définie dans une classe `A` depuis une autre classe `B`, il faut la désigner par `A.f`.

### Question 4.4

1. Un paquetage permet de regrouper des classes concernant un même domaine d'intérêt.
2. L'usage de paquetages n'apporte rien quant à la modularité.
3. L'usage de paquetages permet un niveau de modularité plus vaste que les classes.
4. Les paquetages permettent de structurer les bibliothèques d'intérêt général.
5. La variable d'environnement `CLASSPATH` indique (au compilateur ou à l'exécuteur de programmes java) dans quels répertoires se trouvent les classes et les paquetages à utiliser.
6. L'option `-classpath` des commandes `javac` et `java` indique (au compilateur ou à l'exécuteur de programmes java) dans quels répertoires se trouvent les classes et les paquetages à utiliser.
7. Pour utiliser les classes d'un paquetage `ppp`, on place généralement en début du texte source une directive `import ppp.*;`
8. Pour utiliser les classes d'un paquetage `ppp`, il est nécessaire de placer en début du texte source une directive `import ppp.*;`
9. Pour utiliser une classe `A` d'un paquetage `ppp`, on peut la désigner par `ppp.A`.



## Exercice 5

### Question 5.1

Considérons les fonctions :

```
static int choix(boolean cond, int k1, int k2){
    // résultat : k1 si cond, k2 sinon
    if(cond){
        return k1;
    } else {
        return k2;
    }
}

static int f(int k1, int k2){
    // résultat : ???
    return choix(k1 > k2, k1, k2);
}

static int g(int k1){
    // résultat : ???
    return choix(k1 >= 0, k1, -k1);
}

static int h(int k1, int k2){
    // résultat : ???
    return choix(k1 > k2, k1 - k2, k2 - k1);
}
```

1. Le résultat de la fonction  $f(12,56)$  vaut 44.
2. Le résultat de la fonction  $f(56,12)$  vaut -44.
3. Le résultat de la fonction  $f(12,56)$  vaut 56.
4. Le résultat de la fonction  $f$  est la somme de  $k1$  et  $k2$ .
5. Le résultat de la fonction  $f$  est la différence de  $k1$  et  $k2$ .
6. Le résultat de la fonction  $f$  est le maximum de  $k1$  et  $k2$ .
7. Le résultat de la fonction  $f$  est la valeur absolue de la différence de  $k1$  et  $k2$ .
8. Le résultat de la fonction  $g$  est  $k1$ .
9. Le résultat de la fonction  $g$  est -1 si  $k1 \geq 0$ , +1 sinon.
10. Le résultat de la fonction  $g$  est la valeur absolue de  $k1$ .
11. Le résultat de la fonction  $g$  est  $k1$  si  $k1 \geq 0$ , - $k1$  sinon.
12. Le résultat de la fonction  $h(12,56)$  vaut 44.
13. Le résultat de la fonction  $h(56,12)$  vaut -44.
14. Le résultat de la fonction  $h(12,56)$  vaut 56.
15. Le résultat de la fonction  $h$  est la somme de  $k1$  et  $k2$ .
16. Le résultat de la fonction  $h$  est la différence de  $k1$  et  $k2$ .
17. Le résultat de la fonction  $h$  est le maximum de  $k1$  et  $k2$ .
18. Le résultat de la fonction  $h$  est la valeur absolue de la différence de  $k1$  et  $k2$ .

**Question 5.2**

Considérons la fonction :

```
static String choix(int c, String s1, String s2, String s3){  
    // pré-requis : ???  
    // résultat : ???  
    if(c == 1) {  
        return s1;  
    } else if (c == 2) {  
        return s2;  
    } else /*c == 3*/ {  
        return s3;  
    }  
}
```

1. Il y a des erreurs de syntaxe.
2. `choix(2,"alfred","toto","dupont")` vaut "alfred".
3. `choix(2,"alfred","toto","dupont")` provoque une erreur à l'exécution.
4. `choix(2,"alfred","toto","dupont")` vaut "toto".
5. `choix(5,"alfred","toto","dupont")` provoque une erreur à l'exécution.
6. `choix(5,"alfred","toto","dupont")` vaut "dupont".
7. Les commentaires de spécification :

```
// pré-requis : c >= 1 et c <= 3  
// résultat : s1, s2 ou s3 selon que c vaut 1, 2 ou 3
```

sont acceptables pour cette fonction.

8. Les commentaires de spécification (sans pré-requis) :

```
// résultat : s1, s2 ou s3 selon que c vaut 1, 2 ou 3
```

sont acceptables pour cette fonction.

9. Les commentaires de spécification (sans pré-requis) :

```
// résultat : s1 si c vaut 1, s2 si c vaut 2,  
// s3 pour toute autre valeur de c
```

sont acceptables pour cette fonction.

10. Les commentaires de spécification :

```
// pré-requis : c >= 1 et c <= 3  
// effet : affiche s1, s2 ou s3 selon que c vaut 1, 2 ou 3
```

sont acceptables pour cette fonction.

11. La fonction `choix` pourrait être programmée en utilisant l'aiguillage suivant :

```
static String choix(int c, String s1, String s2, String s3) {  
    switch(c){  
        case 1 : return s1;  
        case 2 : return s2;  
        default : return s3;  
    }  
}
```

12. La fonction choix ne peut pas être programmée par l'aiguillage précédent car c'est une erreur de syntaxe.
13. La fonction choix ne peut pas être programmée par l'aiguillage précédent car l'absence d'instructions break dans les branches rend le résultat incorrect.

### Question 5.3

Considérons la fonction :

```
static boolean comprisEntre(int k, int a, int b){  
    // pré-requis : a <= b  
    // résultat : indique si a <= k <= b  
    boolean resul;  
    if (k < a) {  
        resul = false;  
    }  
    if (k > b) {  
        resul = false;  
    } else {  
        resul = true;  
    }  
    return resul;  
}
```

1. Il y a des erreurs de syntaxe.
2. Cette fonction est incorrecte (le résultat ne correspond pas à la spécification).
3. Cette fonction est correcte.
4. comprisEntre(2,14,56) vaut true.
5. comprisEntre(2,14,56) vaut false.
6. comprisEntre(62,14,56) vaut true.
7. comprisEntre(62,14,56) vaut false.

**Question 5.4**

Considérons la fonction :

```
static int valeurDeCarte(String c){  
    // pré-requis : c vaut "valet", "dame", "roi" ou "as"  
    // résultat : valeur de la carte de nom c (respectivement 1, 2, 3 et 10)  
    switch(c) {  
        case "valet" : return 1;  
        case "dame" : return 2;  
        case "roi" : return 3;  
        default : return 10;  
    }  
}
```

1. Il y a des erreurs de syntaxe.
2. Il y a des erreurs de syntaxe pour certaines versions de Java.
3. Cette fonction peut provoquer une erreur lors de l'exécution.
4. Cette fonction peut rendre un résultat incorrect (résultat qui ne correspond pas à la spécification).
5. Cette fonction est correcte.

## Exercice 6

### Question 6.1

Considérons les fonctions :

```
public static double p1(double x, int k){
    // pré-requis : k >= 0
    // résultat : ???
    double resul = 1;
    int i = 0;
    while(i < k){
        resul = resul * x;
        i++;
    }
    return resul;
}

public static double p2(double x, int k){
    // pré-requis : k >= 0
    // résultat : ???
    if (k == 0) {
        return 1;
    } else {
        return x * p2(x, k - 1);
    }
}
```

1. Les fonctions p1 et p2 sont équivalentes (rendent des résultats identiques).
2. Les fonctions p1 et p2 ne sont pas équivalentes (ne rendent pas des résultats identiques).
3. La fonction p1 est récursive.
4. La fonction p2 est récursive.
5. La fonction p1 est itérative.
6. La fonction p2 est itérative.
7. p1(10,5) vaut 50.
8. p2(10,5) vaut 50.
9. p1(10,5) vaut 100000.
10. p2(10,5) vaut 100000.
11. p1(2,5) vaut 10.
12. p2(2,5) vaut 32.
13. p1(1,5) vaut 5.
14. p2(1,5) vaut 1.
15. p1(-1,5) vaut -5.
16. p2(1,5) vaut -1.

17. La fonction p1 rend en résultat x fois k.
18. La fonction p1 rend en résultat x à la puissance k (x multiplié k fois par lui-même).
19. La fonction p1 ne termine pas toujours.
20. La fonction p2 rend en résultat x fois k.
21. La fonction p2 rend en résultat x à la puissance k (x multiplié k fois par lui-même).
22. La fonction p2 ne termine pas toujours.

### Question 6.2

Considérons la fonction :

```
static int log(int b,int n){
    // pré-requis : b >= 2, n > 0
    // résultat : la partie entière du logarithme base b de n
    int resul = 0;
    int nn = n;
    while(nn != 1){
        nn = nn / b;
        resul++;
    }
    return resul;
}
```

*Rappel : la partie entière du logarithme base b de n est égal au nombre de chiffres significatifs de l'écriture en base b de n moins 1. On doit avoir par exemple  $\log(10, 3675) = 3$ , car  $1000 \leq 3675 < 10000$ .*

1.  $\log(10,10000)$  rend 4.
2.  $\log(10,10000)$  rend 1.
3.  $\log(10,10000)$  ne termine pas.
4.  $\log(2,4)$  rend 2.
5.  $\log(2,32)$  rend 5.
6.  $\log(2,32)$  ne termine pas.
7.  $\log(10,3675)$  rend 3.
8.  $\log(10,3675)$  ne termine pas.
9. Cette fonction est correcte (satisfait sa spécification).
10. Cette fonction est incorrecte car ne termine pas toujours.

## Exercice 7

### Question 7.1

On souhaite définir une structure qui regroupe les caractéristiques de chaque employé d'une entreprise. Ces caractéristiques sont le **nom**, le **prénom**, et la **nature du poste** occupé. Les natures de poste sont **technique**, **administratif** ou **commercial**. La nature du poste est une valeur du type énuméré ainsi défini :

```
enum NatureDePoste {technique, administratif, commercial};
```

1. Une bonne définition de la structure envisagée est :

```
class Employe{
    public static String nom;
    public static String prenom;
    public static NatureDePoste emploi;
    public Employe(String n, String p, NatureDePoste e){
        nom = n;
        prenom = p;
        emploi = e;
    }
}
```

2. Une bonne définition de la structure envisagée est :

```
class Employe{
    public String nom;
    public String prenom;
    public NatureDePoste emploi;
    public Employe(String n, String p, NatureDePoste e){
        nom = n;
        prenom = p;
        emploi = e;
    }
}
```

3. Pour créer un objet de type Employe et capter sa référence par l'identificateur durand, il faut écrire :

```
Employe durand = new Employe("Durand","Alfred",NatureDePoste.commercial);
```

4. Pour créer un objet de type Employe et capter sa référence par l'identificateur durand, il faut écrire :

```
Employe("Durand","Alfred",NatureDePoste.commercial) durand;
```

5. Avec la (bonne) définition de la classe Employe, et la (bonne) création de durand, le nom de durand s'obtient par l'expression :

```
durand.nom
```

6. Avec la (bonne) définition de la classe Employe, il est impossible de modifier la nature du poste d'un employé.

7. Avec la (bonne) définition de la classe `Employe`, on peut modifier la nature du poste d'un employé. Pour attribuer à `durand` la nature de poste administratif, il suffit d'exécuter :  
`durand = Employe(administratif);`
8. Avec la (bonne) définition de la classe `Employe`, on peut modifier la nature du poste d'un employé. Pour attribuer à `durand` la nature de poste administratif, il suffit d'exécuter :  
`durand.emploi = NatureDePoste.administratif;`

### Question 7.2

La structure suivante décrit les caractéristiques d'un produit vendu dans un magasin :

```
class DescriptifProduit{// descriptif d'un produit
    public String nomProduit;
    public String nomFournisseur;
    public double prixDAchat;
    public double tauxTVA;
    public double margeBeneficiaire;
    public DescriptifProduit(String nP, String nF,
        double pA, double tva, double m) {
        nomProduit = nP ;
        nomFournisseur = nF ;
        prixDAchat = pA ;
        tauxTVA = tva ;
        margeBeneficiaire = m;
    }
}
```

1. Pour créer le descriptif d'une lessive, on peut écrire :  
`new DescriptifProduit("lessiveMachin","machin",12,19.6,0.2)`
2. Pour créer le descriptif d'une lessive, on peut écrire :  
`DescriptifProduit lessiveMachin("lessiveMachin","machin",12,19.6,0.2)`
3. Pour créer le descriptif d'une lessive et capter sa référence par la variable `lessiveMachin` on peut écrire :  
`DescriptifProduit ("lessiveMachin","machin",12,19.6,0.2) lessiveMachin;`
4. Pour créer le descriptif d'une lessive et capter sa référence par la variable `lessiveMachin` on peut écrire :  
`DescriptifProduit lessiveMachin =`  
`new DescriptifProduit("lessiveMachin","machin",12,19.6,0.2);`
5. Le nom du fournisseur de `lessiveMachin` s'obtient par l'expression :  
`nomFournisseur(lessiveMachin)`
6. Le nom du fournisseur de `lessiveMachin` s'obtient par l'expression :  
`lessiveMachin.nomFournisseur`
7. La fonction suivante :



```
static double prixDeVente(DescriptifProduit d){  
    double prixHorsTaxe = prixDAchat * (1 + margeBeneficiaire);  
    return prixHorsTaxe * (1 + tauxTVA);  
}
```

est syntaxiquement correcte et rend en résultat le prix de vente du produit décrit par d.

8. La fonction suivante :

```
static double prixDeVente(DescriptifProduit d){  
    double prixHorsTaxe = d.prixDAchat * (1 + d.margeBeneficiaire);  
    return prixHorsTaxe * (1 + d. tauxTVA);  
}
```

est syntaxiquement correcte et rend en résultat le prix de vente du produit décrit par d.

## Exercice 8

### Question 8.1

On considère les déclarations et créations suivantes de chaînes de caractères :

```
String s1 = "tant va la";  
String s2 = "cruche à l'eau";  
String s3 = s1 + s2;  
String s4 = s1 + " " + s2;  
String s5 = s1 + "\"" + s2;
```

1. L'expression `s3=="tant va la cruche à l'eau"` vaut true.
2. L'expression `s4=="tant va la cruche à l'eau"` vaut true.
3. L'expression `s5=="tant va la cruche à l'eau"` vaut true.
4. L'expression `s3.equals("tant va la cruche à l'eau")` vaut true.
5. L'expression `s4.equals("tant va la cruche à l'eau")` vaut true.
6. L'expression `s5.equals("tant va la cruche à l'eau")` vaut true.
7. L'expression `s1.substring(0,3)=="tan"` vaut true.
8. L'expression `s1.substring(0,3)=="tant"` vaut true.
9. L'expression `s1.substring(0,3).equals("tan")` vaut true.
10. L'expression `s1.substring(0,3).equals("tant")` vaut true.

### Question 8.2

Soit la déclaration et création de chaîne :

```
String s = "abcd";
```

1. La longueur de `s` est donnée par l'expression `s.length`.
2. La longueur de `s` est donnée par l'expression `s.length()`.
3. La longueur de `s` est donnée par l'expression `length(s)`.
4. L'expression `s.charAt(2)` vaut le caractère 'b'.
5. L'expression `s.charAt(2)` vaut le caractère 'c'.
6. L'objet chaîne de caractères désigné par `s` est modifiable.
7. L'instruction `s.charAt(2)='x'` ; transforme la chaîne désignée par `s` en "axcd".
8. L'instruction `s.charAt(2)='x'` ; transforme la chaîne désignée par `s` en "abxd".
9. `s.charAt(2)='x'` ; est une erreur de syntaxe.
10. L'instruction `s=s.substring(0,2)+'x'+s.substring(3,4)` ; crée la chaîne "axcd" et la fait désigner par `s`.
11. L'instruction `s=s.substring(0,2)+'x'+s.substring(3,4)` ; crée la chaîne "abxd" et la fait désigner par `s`.
12. `s=s.substring(0,2)+'x'+s.substring(3,4)` ; est une erreur de syntaxe.
13. `s=s.substring(0,2)+'x'+s.substring(3,4)` ; provoque une erreur à l'exécution.

## Exercice 9

### Question 9.1

1. Les tableaux servent à faire correspondre des nombres entiers (les indices) et des données de types quelconques mais identiques (les éléments du tableau).
2. Un tableau remplace avantageusement l'usage de variables de même type, car il est plus concis d'utiliser des noms de la forme `T[0]`, `T[1]`, `T[2]`, `T[3]` ... que des identificateurs divers.
3. En Java, les tableaux sont des objets toujours désignés par référence.
4. En Java, on peut créer un tableau de 20 éléments (de type `int` par exemple) par une simple déclaration de la forme : `int[20] T;`
5. En Java, un tableau de 20 éléments (de type `int` par exemple) est nécessairement créé par une instruction : `new int[20];`
6. En Java, une déclaration de la forme `int[] T;` définit `T` comme étant une référence à un tableau d'entiers et cette référence est initialisée à `null`, de sorte que `T` ne désigne rien.
7. En Java, `T2` étant déclaré `int[] T2;` et `T1` désignant un tableau d'entiers de taille 30, l'instruction d'affectation `T2 = T1;` provoque la création d'un tableau de taille 30 dont la référence est captée par `T2` et la copie des éléments de `T1` dans le tableau désigné par `T2`.
8. En Java, `T2` étant déclaré `int[] T2;` et `T1` désignant un tableau d'entiers, l'instruction d'affectation `T2 = T1;` provoque le captage par `T2` de la référence au tableau désigné par `T1`.

### Question 9.2

Soit la séquence d'instructions :

```
String[] T1 = new String[30];  
String[] T2 = T1;  
T2[4] = "bonjour";
```

1. Après exécution de cette séquence, `T1[4]` vaut `null`.
2. Après exécution de la séquence précédente, `T1[4]` vaut la chaîne vide `""`.
3. Après exécution de la séquence précédente, `T1[4]` vaut `"bonjour"`.
4. Après exécution de la séquence précédente, `T1[4]` n'a pas de valeur définie.

**Question 9.3**

On considère les fonctions suivantes :

```
static void renverseLOrdre(int[] T){
    for(int i = 0; i < T.length / 2; i++){
        int v = T[i];
        T[i] = T[T.length - i - 1];
        T[T.length - i - 1] = v;
    }
}

static int[] ordreInverse(int[] T){
    int[] resul = new int[T.length];
    for(int i = 0; i < T.length; i++){
        resul[i] = T[T.length - i - 1];
    }
    return resul;
}
```

1. La fonction `renverseLOrdre` modifie le tableau désigné par le paramètre `T`.
2. La fonction `renverseLOrdre` rend en résultat un nouveau tableau contenant les mêmes éléments que `T` dans l'ordre inverse de celui où ils figurent dans `T`.
3. La fonction `renverseLOrdre` renverse l'ordre des éléments de `T`, c'est-à-dire : échange `T[i]` avec `T[T.length-i-1]` pour tout `i` de 0 à `T.length-1`.
4. La fonction `ordreInverse` modifie le tableau désigné par le paramètre `T`.
5. La fonction `ordreInverse` rend en résultat un nouveau tableau contenant les mêmes éléments que `T` dans l'ordre inverse de celui où ils figurent dans `T`.
6. La fonction `ordreInverse` renverse l'ordre des éléments de `T`, c'est-à-dire : échange `T[i]` avec `T[T.length-i-1]` pour tout `i` de 0 à `T.length-1`.

**Question 9.4**

1. En Java, on peut créer un tableau de taille nulle, par exemple un tableau d'entiers de zéro éléments, par `new int[0]`.
2. Un tableau de taille nulle est désigné par `null`.
3. Considérons un tableau de tableau ainsi déclaré : `double[][] T;`.
4. La taille des toutes les lignes de `T` s'obtient par `T.length`.
5. La taille de toutes les lignes de `T` s'obtient par `T[0].length`.
6. Si `T` est un tableau régulier rectangulaire et si `T.length != 0`, la taille de toutes les lignes de `T` s'obtient par `T[0].length`.
7. Si  $i \geq 0$  et  $i < T.length$ , la taille de la ligne `i` de `T` s'obtient par `T[i].length`.